

University of Bamberg

Distributed Systems Group



Master Thesis

in the degree programme International Software Systems Science
at the Faculty of Information Systems and Applied Computer Sciences,
University of Bamberg

Topic:

Adapting Stream-Based State-Machine Replication to Apache Flink

Author:

Md Moinuddin

Reviewer:
Prof. Dr.-Ing. habil. Tobias Distler

Date of submission:
03.08.2026

Contents

1 Introduction **1**

1.1 Motivation and Problem Context 1

1.2 Scope and Boundaries of This Work 2

2 Background and Foundational Concepts **3**

2.1 State-Machine Replication (SMR) Principles 3

2.2 Consensus in Fault-Tolerant Systems 4

2.3 Distributed Stream Processing: Concepts and Challenges 5

2.4 Apache Flink: Dataflows and Stateful Processing 6

2.5 Architecture of TARA 7

3 Problem Statement and Research Approach **9**

3.1 Key Challenges in Stream-Based Coordination 9

3.2 System Requirements and Properties 9

3.3 Strategy for TARA Implementation on Flink (Gumti) 10

4 Gumti: Design and Implementation in Apache Flink **12**

4.1 System Overview and Dataflow Topology 12

4.2 End-to-End Request Processing Pipeline 13

4.2.1 Client Request Ingestion and Batching 13

4.2.2 Leader-Driven Sequence Number Assignment 14

4.2.3 Commit Phase and Broadcasting 14

4.2.4 Request Execution and Result Delivery 15

4.3 Garbage Collection 15

4.3.1 Flink Checkpoint for Durability 15

4.3.2 Progress Tracking and State Pruning 16

4.4 View Change Mechanism 16

4.4.1 Leader Liveness Monitoring via Timers 16

4.4.2 View Change Propagation through Streams 17

4.4.3 State Reconstruction from Durable Logs . . . 17
4.4.4 Resuming under the New Leader . . . 17
4.5 Gumti Architecture and Flink Integration . . . 18
4.5.1 From Tara Nodes to Flink Operators . . . 18
4.5.2 How Replicas Interact with the Flink Runtime . . . 18
4.5.3 Flink Services Used by Gumti . . . 19
4.6 Flink Cluster Deployment and Operator Parallelism . . . 20
4.7 Core Data Types and Communication Primitives . . . 22
4.8 Key Adaptations from Original Tara Design . . . 23
4.9 Technical Challenges and Engineering Solutions . . . 24
4.10 Useful Flink Features but Not Used . . . 25
4.11 Summary of Design Decisions . . . 26

5 Experimental Assessment **28**
5.1 Evaluation Objectives . . . 28
5.2 Experimental Setup and Environment . . . 28
5.3 Development Complexity and Code Characteristics . . . 29
5.4 Performance: Throughput, Latency . . . 29
5.4.1 Batched Consensus . . . 29
5.4.2 Non-Batched Consensus . . . 30
5.4.3 Influence of the Engine's Buffering . . . 31
5.5 Fault Tolerance and Recovery Experiments . . . 31
5.6 Discussion of Experimental Findings . . . 32

6 Related Work **33**
6.1 Consensus and Byzantine Fault Tolerance . . . 33
6.2 Stream Processing Systems . . . 33
6.3 Data Flow as a Unifying Concept . . . 35
6.4 Positioning Gumti in the Research Landscape . . . 36

7 Conclusion **37**

7.1 Summary of Key Contributions 37

7.2 Limitations 38

References **39**

List of Figures

1 State-machine replication mechanism 3
2 Flink process model (Taken from Apache flink [3] paper) 7
3 Overview of the Tara replication protocol (Taken from Tara [9] paper) . . . 8
4 Overview of Gumti protocol 12
5 Batched consensus with buffer timeout 0 30
6 Batched consensus with buffer timeout 2ms 30
7 Non-batched consensus with buffer timeout 0 30
8 Non-batched consensus with buffer timeout 2ms 30
9 Gumti's fault tolerance 31

List of Tables

1 Flink services used by Gumti 19

2 Operator replication levels in Gumti (conceptual design) 20

3 Batching parameters in request source 22

4 Mapping of Tara nodes to Flink implementation components 22

Listings

Abbreviations

1 Introduction

This thesis studies how the Tara replication protocol, originally built on Apache Heron [6], can be rebuilt on Apache Flink [3]. The Flink-based realization of Tara developed in this work is called Gumti. This chapter explains what problems are there regarding this topic and how I want to solve that.

1.1 Motivation and Problem Context

ault tolerance in a distributed service usually begins with a single choice: running multiple copies of the service. This is what state-machine replication (SMR) [12] is about.The concept is that you have multiple copies of your service running at once and all those copies receive the same series of commands in the same order. If they all start from the same state and apply the same commands, they remain synchronized, and the service continues to operate despite crashing some copies.

It is hard to decide which command to execute first. That is where a consensus protocol comes into play. Protocols such as Paxos [8] and Raft [11] need to deal with issues of selecting the leader, dealing with duplicate proposals, deleting outdated data and recovering from crashes. There are lots of concurrent code runs for these protocols; therefore, even minor errors can turn into bugs that are very hard to find.

The other approach is that of Tara [9]. It does not write a replication library with its own networking, threads or storage; rather it constructs the protocol as a flow of data. Every step is a small node that does a single job and client commands are passed from one node to another. Since it is built using a stream processing engine, Tara makes use of the functionality that the stream processing engine already provides, such as routing, state storage and snapshot recovery. Because of this it requires less code and is still fast. Tara has been built and tested in Apache Heron.

This leads to a simple question. Does the idea only work on Heron, or does it work on other engines too? Because Heron is not the preferred option anymore. Apache Flink [3] is a popular open source stream-processing engine that is widely used in industry, such as large-scale projects by Alibaba, Netflix and Uber [13]. If it also works with Flink, it is not limited to a single engine and for many more people, building a fault-tolerant service becomes easier.

Moving Tara [9] to Flink [3] is not just a copy job, as there are differences in the behaviour of the two engines. In Heron [6] a node can broadcast a message to a set of selected downstream tasks. The keyBy operation in Flink only sends each key to one instance and there is no built-in way of sending the message to all replicas. State and recovery work also different. These kinds of gap make the work interesting, because they show which parts of the idea are general and which parts are depending on the engine.

Because of this problem, the thesis is reimplementing Tara on Apache Flink. This new system is called Gumti. To build it, we use alternate features of Heron that are equal in Flink. The thesis has three goals. One is to show the ability of Gumti to maintain the correct behaviour of Tara and its fault tolerance. Second, to measure its throughput and latency. And third, to use these results to see which parts of Tara's design are general,

and which parts depend on the engine.

1.2 Scope and Boundaries of This Work

A few limits keep the work focused. Gumti has only crash faults, just like Tara. Byzantine behaviour is not covered because a replica will not send wrong information, it will just stop. Gumti is also 2F + 1 replica that can withstand F crash failures, similar to Tara [9]. For this evaluation, it has been set to F = 1 (three replicas) as is done in experiments for Tara.

The evaluation runs on a single machine rather than a cluster, so the performance numbers should be read in that light. The goal is to observe how the system behaves and can we reproduce the results like Tara's. The real contribution is to show how we move Tara's design to Flink - what we did not change, what has to change, and how well it performs.

The rest of the thesis is organised as follows. Chapter 2 covers the background: state-machine replication, consensus, stream processing, Apache Flink, and the design of Tara. Chapter 3 shows the problem more precisely, lists the core requirements for the implementation, and explains the plan for implementing Gumti. Chapter 4 describes the design and implementation in detail, including the request pipeline, garbage collection, view change, the mapping of Tara nodes to Flink operators, and the main adaptations and challenges. Chapter 5 shows the experiments of Gumti's development effort, performance, and fault tolerance. Chapter 6 shows related work, and Chapter 7 concludes with a summary, the limitations.

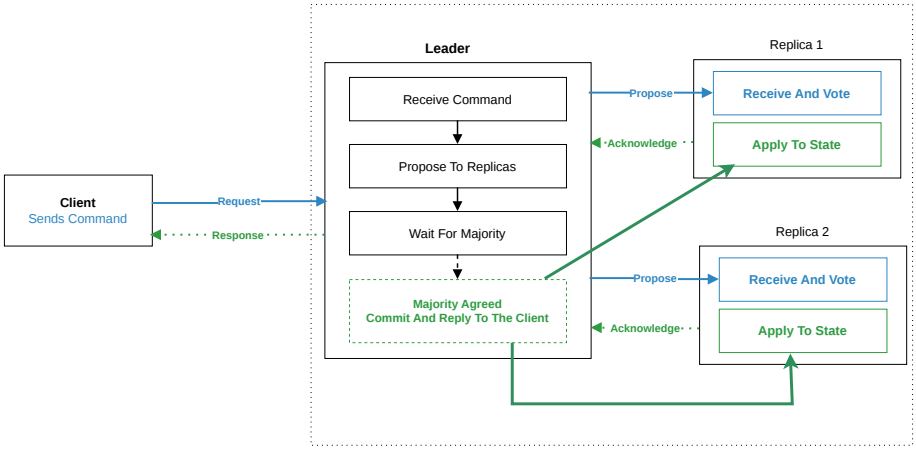


Figure 1: State-machine replication mechanism

2 Background and Foundational Concepts

In this section we will discuss the principles of state-machine replication, what is consensus in fault-tolerant systems, their concepts and challenges and a brief summary of Apache Flink [3] and Tara [9].

2.1 State-Machine Replication (SMR) Principles

State-machine replication (SMR) is one of the most common ways to make a service fault-tolerant [12] by keeping several copies of the same application running on different machines (Figure 1). The concept of state-machine replication can be defined as follows. The main idea is quite simple. As long as all the working replicas get the same list of client requests in the same order and process them in exactly the same way, they will all end up with the same state and send back the same answers to clients, even if some of the replicas crash.

However, for this to function correctly, two factors must be true. First, The state machine must be 100% deterministic, that means the same input in the same state always results in the same output, the same next state, regardless of the delay of the messages, or the order of the threads. Second, the order of the application of the commands has to be the same across all replicas. Without this total order, you quickly start getting inconsistent states across the system [7].

Many SMR systems, like the one we will discuss in this thesis, assume that the only type of fault is a crash fault, meaning that a replica may fail completely but will not send incorrect information. A system which has n replicas, you can usually handle up to f failures as long as $n \geq 2f + 1$. This follows the majority rule, where a decision (for example, accepting a proposal or committing a log entry) must be agreed upon by more than half of the replicas. Note that any two groups, each containing more than half of the replicas, will always overlap meaning they will have at least one replica in common, and this is exactly what keeps the system progress safely, as long as a majority stays correct, there is always a consistent view of what has been decided.

There are two large goals: strong consistency and availability. Strong consistency means that when one node updates a value, all other nodes will eventually see this update in a consistent global order. Availability means that the system will still be able to respond to queries even if some of the machines are down, as long as enough of them remain running.

Replicas maintain two items in order to handle the recovery from a crash: a log containing every command that has been committed and carried out, and occasionally, a checkpoint, which is a current state of the application. Both the log and the checkpoints are stored in persistent memory, so they survive crashes. When a replica restarts, it reloads its local log and last checkpoint, then requests only the missing commands from other replicas. It replays these operations from the last checkpoint onward to catch up. This approach to recovery is reasonably efficient and ensures that no committed command is ever lost.

In Paxos [8] or Raft [11], a leader is elected and proposes the next command. It has to get acknowledgment from the majority of replicas before proceeding with the command. Although the leader-based approach makes coordination easy and has good performance in stable conditions, the leader can be become a performance bottleneck when there is a heavy load on the system, and the process of electing a leader results in additional overhead during leader changes.

Both Tara [9] and our Gumti use determinism, total ordering, majority quorums, crash tolerance, and logging for recovery. In other words, Tara uses the stream engine's timers and state backends to arrange and recover from crashes in order to run the replication as a continuous dataflow.

The transition from the traditional request-response based consensus to continuous stream-based consensus comes with its benefits and also its drawbacks, so we will take a closer look at the consensus protocols in the next sections and then, look into how the distributed stream processing system, Apache Flink, works.

2.2 Consensus in Fault-Tolerant Systems

The most popular consensus algorithm is Paxos [8], and it was first proposed by Leslie Lamport. In this, the nodes are divided into three roles: proposer, acceptor, and learner. In the Paxos algorithm, during the leader election phase, a proposer sends a Prepare message containing a ballot number to the acceptors. The goal is to become the leader for a particular slot and to find out what the acceptors have already promised. If the proposer receives enough responses, then it sends an Accept message to the acceptors with the value it wants them to agree on. Once the majority has been achieved, the value is considered to be chosen, and the learners can observe the value. The Paxos consensus algorithm is guaranteed to be correct if the majority is non-faulty, but it is perhaps the most difficult consensus algorithm to implement due to the various complexities of the leader changing and the possibility of duplicate proposals.

Later on, Diego Ongaro and John Ousterhout designed Raft [11] precisely because they wanted something that people could understand and implement more easily. Raft is a leader-based approach from the beginning because nodes will start an election using random timeouts. This means that the node that receives the majority vote will become the leader and start adding client commands to a replicated log. The leader will then

push the log entries to the followers, and as soon as it hears from the majority, it will commit the entry and notify everyone. In addition, the follower's logs are ensured to be matching the leader's using the log matching rule in Raft. Moreover, the separation is clear between the different operations here, and that is why it is used so frequently.

The sections below provide more information on this topic, explaining distributed stream processing and why its core mechanisms are well-suited for implementing state-machine replication.

2.3 Distributed Stream Processing: Concepts and Challenges

Distributed stream processing [3] is a paradigm for the processing of continuous, unbounded streams of events. Unlike batch processing, which requires the processing of complete datasets before starting, stream processing treats the data as an endless stream. Events occur continuously, or at least in bursts, and the processing system responds immediately, executing computations via a directed cyclic graph (DAG) of operators. Operators apply simple operations such as filtering, mapping, aggregating, or joining, propagating the results to other operators, executing the entire DAG in parallel across the cluster.

In data-stream processing [3], when any node crashes happen or tasks restart then you need a way to recover. But recovering is not just about restarting; it is about figuring out what to do with all the events that were halfway through being processed when things went wrong. Most stream systems need to remember things as data flows through, such as a running total, a count of events within a certain time window, or a table of user IDs and their most recent actions. Each worker has a local copy of that memory, so that it survives a restart. When a worker crashes, all that saved-up memory needs to be restored before it can pick up where it left off.

When a failure occurs, what happens to the data that was being processed at that moment? To handle this, there are three possible ways. The simplest approach is to just ignore any events that were being processed when the crash happened, the system restarts fresh and moves on. You will not see duplicate data, but whatever was mid-processing will be lost. Another approach is to re-send any events that were being processed at the time of the crash, so that no data gets lost. The problem is that some events might get processed twice, once before the crash and once after the recovery, which can cause incorrect results. The ideal method is exactly-once processing [3]: if there is a crash in the middle, each event is processed exactly once. There will be no loss and no double counting. It is very important in systems where precision is essential, such as banking transactions or audit logs, where even a single duplicate or missing entry can cause serious issues. To make this work, the system frequently takes a snapshot, and it saves a snapshot of the state of each worker every so often. It also transmits special messages called barriers through the pipeline, and once all the workers have received the same message, a snapshot is made. If a component crashes, the system will load from last checkpoints and only re-run events that were not completed in time.

Each event passes through the system with a small data record, for example: a timestamp and a few data fields such as a user ID, a transaction number, or a sensor reading. Since many operations need to group related events together, such as counting actions by user, each event typically has a key field that indicates to the system which group it belongs

to.

There are several important mechanisms introduced with distribution. Partitioning, often achieved with a `keyBy` operation in Flink [3], allows events with the same key to go to the same instance of parallel processing while maintaining order within keys. Parallelism defines how many instances of a given operator run across the cluster. Increasing parallelism can improve throughput but increases the shuffling overhead during repartitioning.

These built-in features make stream processing engines well suited for SMR tasks. Traditional SMR relies on consensus mainly to achieve total ordering of commands [7, 12], while recovery uses logs of those ordered commands, checkpoints, and state transfer from peers [11]. Stream engines provide these features natively through dataflows, timers and checkpoints [9], to decrease reliance on an external consensus service.

Popular systems provide different focuses on these building blocks: Apache Storm [14] is focused on topologies that process data with at-least-once delivery and low latency, Heron [9] is focused on reliable and easy-to-use processing, and Flink [3] has solid native support for exactly-once processing and state processing. Through these systems, the logic of replication may be made complex and defined as regular streaming jobs on shared clusters.

Challenges still exist, such as handling out-of-order events, watermarking for event time, checkpointing, and backpressure, but overall, these abstractions seem to map well to a protocol like Tara [9]. In the following subsection, we discuss Apache Flink [3] in particular, as it is the foundation of Gumti.

2.4 Apache Flink: Dataflows and Stateful Processing

Apache Flink [3] is a distributed stream processing engine that unifies batch and stream processing within a single system. No matter what kind of program is written in Flink, whether it is batch processing using the Dataset API or stream processing using the DataStream API, everything is eventually converted into a single dataflow graph (Figure 2). This graph works like a flowchart that clearly defines the order of tasks, where the data comes from, and where it should go after processing. To run this dataflow graph, Flink has three main components. First, the Client is responsible for creating the entire dataflow graph and submitting it to the system. Next, the JobManager acts as the central coordinator. It receives the graph from the client and determines which TaskManager will perform which task, assigns it to the task and constantly monitors the entire process. The JobManager is responsible for identifying problems if any server crashes, and handling recovery. Finally, the TaskManagers are the actual workers that perform the real data processing tasks.

Flink [3] uses buffer timeout to maintain a good balance between latency and throughput. The buffer works like a small container that holds data before sending it to the next operator. If data is sent immediately as soon as it arrives, the latency stays very low, but this creates high network overhead because small messages are sent frequently. On the other hand, if we wait until the buffer is completely full before sending, network cost becomes lower, but latency increases. To solve this trade-off, Flink introduces buffer timeout as a deadline. If the buffer does not fill up within the specified time, the data is

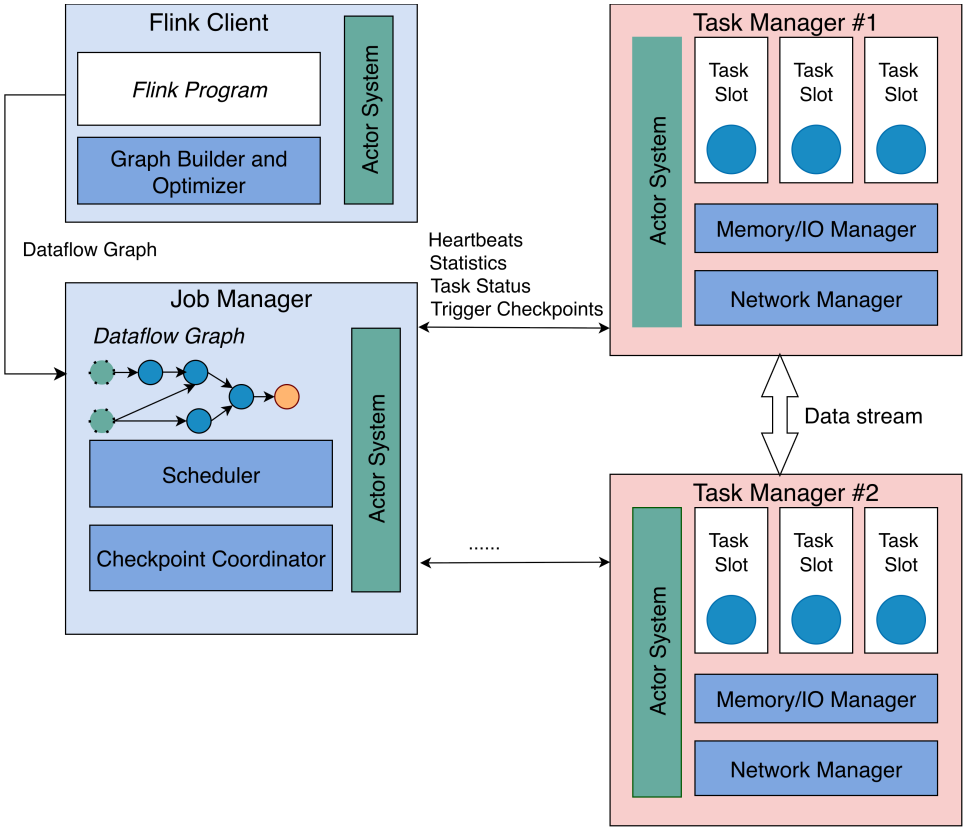


Figure 2: Flink process model (Taken from Apache flink [3] paper)

sent anyway. Users can tune this buffer timeout value based on their specific needs. When failures occur during all of this processing, Flink uses Asynchronous Barrier Snapshotting to ensure fault tolerance. It injects special markers called barriers into the data stream. When an operator receives barriers from all its input streams, it takes a snapshot of its current state and saves it. For this snapshot mechanism, if the system crashes and restarts, it can resume from the latest successful snapshot. This guarantees that no data is lost and no event is processed twice.

2.5 Architecture of TARA

Tara [9] is different to conventional replication systems. Instead of building the protocol as a separate system with its own infrastructure, Tara builds it as a flowing data application that runs on top of a stream processing engine. The main idea is simple, instead of thinking about a set of replicas that need to coordinate with each other, think about how client commands flow through a series of processing stages. Each stage does one specific job, and the data moves from one stage to the next until finally the results come back to the clients.

To build this system, Tara creates several different types of processing nodes, each with a clear job to do. As shown in orange part in the Figure 3, the Request sources ① take incoming commands from clients and pass them to proposer node, which give each command a sequence number to make sure commands are processed in the right order. The proposer send these proposals to committer nodes ②, which save the assignments and confirm them to executor nodes. The executor nodes wait until they hear from enough

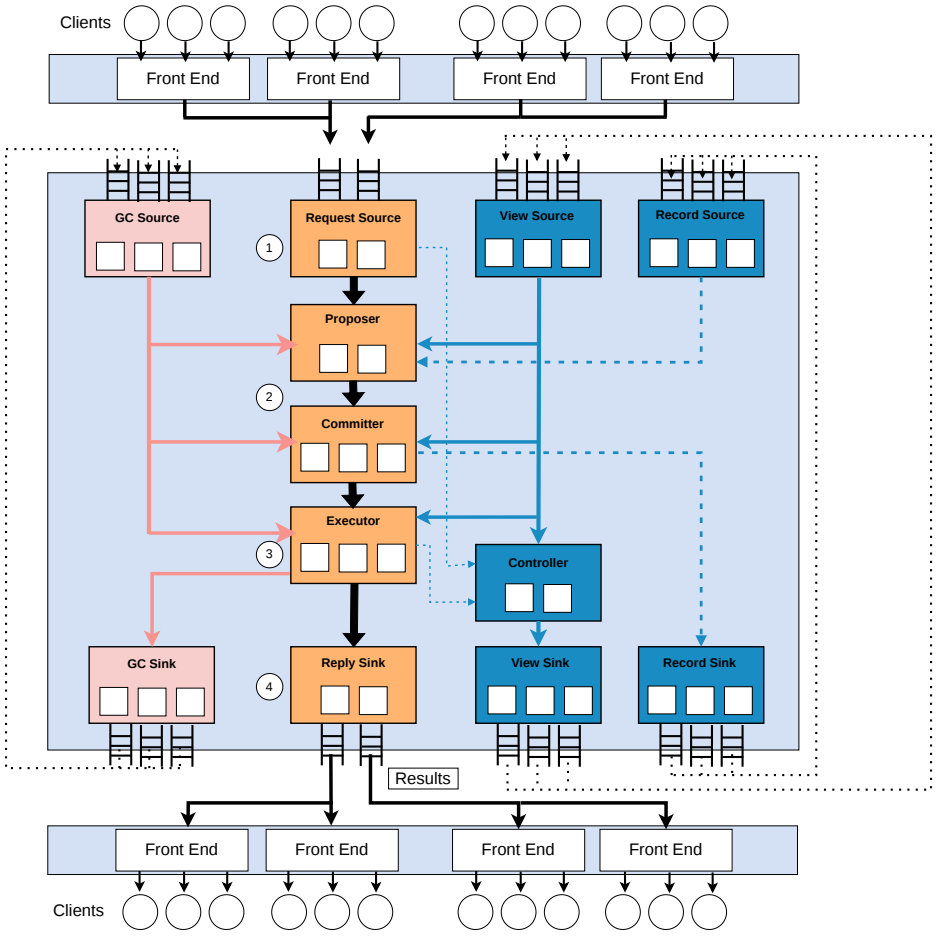


Figure 3: Overview of the Tara replication protocol (Taken from Tara [9] paper)

committers ③, and then they run the command on the actual application and send the results back to the clients ④.

Beyond the core process of handling requests, Tara has other nodes that take care of cleanup and handle failures. In Figure 3 grayish red part, Garbage collection nodes check periodically which requests have been fully executed and stored safely across a sufficient number of replicas, so old information can be thrown away and memory can be freed up. And blue part of the Figure 3, View change nodes watch to make sure everything is moving forward correctly. If the proposer stops working, these nodes notice the problem right away and coordinate switching over to a new leader. In this way, nothing will be lost that has already been processed.

This approach of organizing the work takes advantage of what stream processing engines already provide. As we saw in Section 2.3, stream engines have built-in timers to keep track of what is going on and storage systems to keep things safe, so Tara doesn't have to build all that from scratch. The result is that Tara needs a lot less code than traditional approaches, while still getting good performance. By sending multiple requests through the stages at the same time, it can handle many more commands per second. In the chapters that follow, we will show how we take Tara's ideas and run them on Apache Flink.

3 Problem Statement and Research Approach

Tara [9] is a state-machine replication protocol. It was implemented on Apache Heron [6], but Heron is only one way to run it. The protocol does not depend on Heron, so it could in principle run on other stream engines as well. This leads to the question the thesis starts from: can Tara also run on Apache Flink [3], and what is needed to make that happen? The main difficulty is that Tara expects a few specific things from the engine it runs on, and Flink does not always provide them in the same way. This section sets out that difficulty before the design is shown. It first lists the questions the work has to answer (Section 3.1), then the properties the Flink version, Gumti, has to meet (Section 3.2), and finally the plan for closing the gap between what Tara needs and what Flink offers (Section 3.3).

3.1 Key Challenges in Stream-Based Coordination

Tara was built around what Heron could do. The protocol is based on how its engine routes messages, its failure response and the supporting services it carries. Flink covers some of these needs in a different way. The following three questions narrow down the gaps and why they are important.

RQ1 Can we implement Tara's design on Flink? In Tara, a node emits each tuple to designated receivers: some tuples must reach one specific replica, others must reach all replicas of a stage. On Heron, the framework routes tuples to their intended receivers. Flink routes messages on keyed streams only by hashing a key, and it offers no direct way to address a particular instance or all instances of an operator. This gap throws the question of whether or not Flink's routing is capable of expressing Tara's communication pattern.

RQ2 How replica failure handle by Gumti? Is it similar like Tara? In Tara, when a replica crash, the system still keeps on running. Heron helps here in the original implementation: when a node crashes, Heron starts it again and connects it back to the running system. Flink works differently. When something fails, it restarts the job from its last checkpoint. So the question is whether the recovering approach of Flink's is sufficient for a replication protocol, or does the protocol also need to take on some of the recovery work itself?

RQ3 What replaces a Heron capability that Flink does not offer? Some features that Tara uses might not have an equivalent in Flink. The question is what other mechanism can perform the same task without modifying the protocol.

3.2 System Requirements and Properties

For Gumti to be a faithful version of Tara on Flink, it has to meet the following requirements. They are listed roughly in the order in which they have to be addressed: the

replicas first need to find each other, then they need to exchange messages correctly, and only then can the protocol properties on top of this communication be met.

- **Instance discovery:** Replicas must be able to find each other so they can exchange the messages that travel outside the main dataflow. This comes first because nothing else works without it: as long as a replica does not know where the others are, it cannot open the connections that the feedback loops depend on.
- **Correct message delivery:** Each message must reach the right replica, or every replica, just like the targeted and broadcast sending Tara relies on. This is not only about performance. When a proposal reaches the wrong proposer, it just gets ignored, and a commit message that misses some committers can block agreement, so wrong routing does not slow the protocol down, it stops it.
- **Support for feedback loops:** Tara has parts where output flows back to an earlier stage, for example to announce a new view or to hand over committer records after a leader change. Flink graphs can not contain cycles, but such cycles are allowed to exist outside the graph and still execute properly. If they fail silently, the system survives normal operation but can no longer recover from a fault.
- **Functional equivalence:** Gumti must agree on a single total order for the requests and provide the same safety and liveness guarantees as Tara. In particular, liveness includes fault tolerance: the crash of up to F replicas must not stop the system; it must recover and keep going. Without this, the port would be a stream application that looks like Tara but does not replicate anything.
- **Deterministic execution:** Given the same ordered sequence of requests, every replica must reach the same state. Agreement on the order is only useful if the replicas then also process that order in the same way; otherwise their states drift apart and the replication is worthless.
- **Exactly-once execution:** Each client request must be executed exactly once and produce a single reply, even if the client resends it after a timeout or a leader change. Retries are normal behaviour, not an error case, so a request that gets executed twice would corrupt the application state in a way the client can never see or fix.

3.3 Strategy for TARA Implementation on Flink (Gumti)

The plan is to match each capability Tara needs from its stream engine with the Flink mechanism that can play the same role. The points follow the same order as the requirements in Section 3.2: the structure comes first, then discovery and communication, and finally the fault tolerance that builds on all of it.

- **Dataflow structure:** In Tara, the nodes are one stage of the processing pipeline, and in Heron, they are each a process. The same graph is expressed as a set of flink stream operators; the only exception being the feedback edges—they are handled separately below.

- **Instance discovery:** Part of Tara's communication runs outside the main dataflow, and for that the replicas must know each other's network endpoints, which Flink does not provide on its own. The approach is to add a separate coordination service where each replica registers its endpoint and looks up the others. This has to be in place before the loops below can be closed.

- **Message routing:** In Tara, a node emits each tuple to its designated receivers, either one specific node or all replicas of a stage, and the framework routes the tuples there. Flink does not allow arbitrary routing on keyed streams, but it can partition a stream by a key so that a given key always reaches the same instance. This keyed partitioning can stand in for Tara's targeted delivery, and delivery to all replicas can be achieved by sending one copy of the tuple per replica. These two handle both kinds of routing Tara needs - one-to-one and one-to-all.

- **Feedback loops:** Flink graphs are not allowed to have loops, so Tara's feedback loops have to sit outside the job. The approach is to close them outside the dataflow: a sink sends its output over the network back to a source, using the endpoints from the discovery service, so the loop leaves Flink and re-enters it.

- **Fault tolerance:** Flink can restart a failed operator and bring the job back in running state. Tara's consensus state cannot be restored from a Flink checkpoint; it has to be rebuilt at the protocol level, by electing a new leader and rebuilding the state. The approach combines the two: Operator restart is taken care by Flink, while the protocol handles the consensus-level recovery on top of it.

4 Gumti: Design and Implementation in Apache Flink

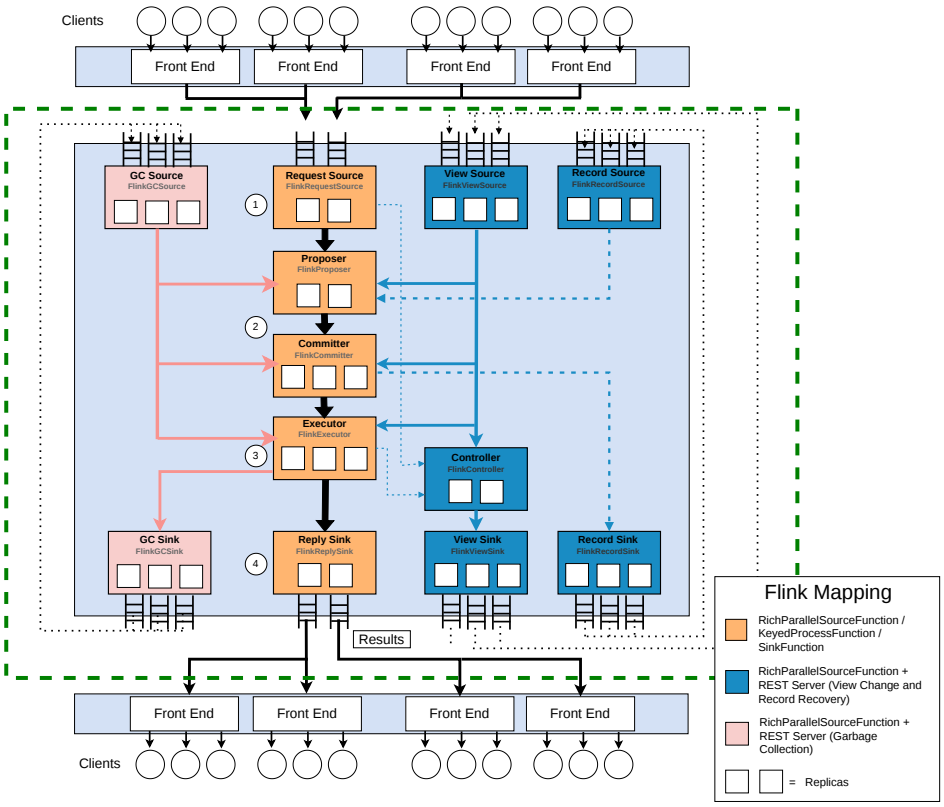


Figure 4: Overview of Gumti protocol

This section describes Gumti's design and implementation in details, showing system overview and dataflow topology 4.1, end-to-end request processing pipeline 4.2, how garbage collection works 4.3, how view change happens 4.4, how we map Gumti and Flink 4.5, Flink cluster deployment and operator parallelism 4.6, core data types and communication primitives 4.7, what are the differences between Gumti and Tara design 4.8, what technical challenges we face during the implementation and the solution 4.9, some useful Flink features 4.10 and the summary of design decisions 4.11.

4.1 System Overview and Dataflow Topology

Gumti stages itself as a suite of operators over Apache Flink coupling them to form the Tara concensus protocol. In Flink, operators are processing functions that 'transform' the incoming data streams. Each role of the protocol is served by a different operator: request sources are the ones that receive the requests from the clients, proposers create the sequence number, committers are responsible for the validation of the decision, and executors apply the request to the state machine. A controller operator is always aware of progress, and is able to identify when a leader fails. The overall dataflow is triggered by four logical sources – clients' requests, view change notifications, previous committed decisions, and garbage collection signals. All the above sources are linked in a directed acyclic graph and the main processing pipeline is: request source, proposer, committer, executor, with controller as a separate monitoring module (as in the figure: 4).

In fact, there can be multiple instances of each operator for a replication purpose. The subset of the operators that participate in the consensus process (the committer and executor) is replicated because it can withstand up to F failures. The proposer is duplicated because of leader failover and the controller is duplicated to ensure controller progress is reliable. In particular, the committer and the executor both have $2F+1$ replicas and if F of them fail, progress is made by a quorum of $F+1$. The proposer is using $F+1$ replicas to keep one leader alive and F standbys. To provide progress monitoring in the face of failures, the controller will rely on $2F+1$ replicas. Additionally, the source operators, namely request source, view source, record source and garbage collection source, are also replicated so that the system can run even if up to F source replicas fail. The routing guarantees that all tuples (for a particular partition) are always sent to the same instance of the operator. This ensures that the state of each partition remains localised and also gives each partition a potential to recover from an incident on one partition.

Figure 4 shows the full architecture of Gumti, which maps each Tara node to Apache Flink operator. The core pipelines are request source, proposer, committer, executor, and reply sink - maps directly to Tara's orange part and it is the the main request processing path of the system. The garbage collection stage (GC source, GC sink) and view change stage (view source, view sink, record source, record sink, controller) correspond to Tara's grayish red and blue part nodes respectively. `RichParallelSourceFunction` is used for data source stage, `KeyedProcessFunction` is used for processing stage and `SinkFunction` is used for data sink.

4.2 End-to-End Request Processing Pipeline

A single request goes through four stages in Gumti: ingestion and batching, assigning sequence number, commit decision, and execution. We describe each stage in detail, showing how requests goes through the operators from client ingestion to result delivery.

4.2.1 Client Request Ingestion and Batching

When clients send requests to a request source operator, it receives the data from the network and batches payloads for efficiency. Each payload includes a client ID, transaction number, and application data. The request source collects multiple payloads into a batch and waits until either the batch reaches a threshold size or a timeout expires. Once a batch is ready, the request source creates a request object containing the batch, and emits a tuple representing this request. Multiple replicas of the request source are running concurrently, with each accepting connections from the clients. Clients can access any replica available, and if one replica fails, the client will try another.

This approach of batching is way more efficient because it reduces the number of data elements flowing through the pipeline. Instead of processing each client payload separately, the system processes them in groups. Batching provides one key benefit: it reduces protocol overhead per client payload because the system operates on batches instead of individual payloads.

4.2.2 Leader-Driven Sequence Number Assignment

When request source emits the request tuples it directly routed to the proposer operator, and it assigns the globally consistent sequence numbers. One proposer must be leader at a time, others will be backup. Leaders are determined by the current view number – specifically, the proposer whose ID matches a formula based on the view number becomes the leader for that view. Each proposer instance applies this formula after the view change, and switches to either active leader mode or backup mode.

In the leader mode, the proposer stores a window of pending requests, a set of records (which is really previous agreed upon commitments from past views), a set of sequence numbers, and the number of the current view. The leader proposer uses the current sequence number for a request tuple when it is sent and forms the consensus tuple: (`Consensus, p, s, v, req`), with variables `p`, `s`, `v`, `req` assigned to the partition number, assigned sequence number, view number, and request, respectively. The tuples of consensus are sent by the proposer to the committer.

The proposer, on receiving a view tuple announcing a view change, updates its current view number. If as a result of the view change, a different proposer instance is elected leader then the old leader ceases to accept any further requests and the new leader starts issuing sequence numbers. This is a handoff to make sure progress continues after a leadership change, as described below in Section 4.4.

Consistent ordering is achieved with this design since all the time the sequence number assignments are performed by a single leader that serializes all the assignments per partition. Backup proposers offer availability - view change election elects one of the backups as a leader which enables the system to continue to serve processing.

4.2.3 Commit Phase and Broadcasting

Proposer generates consensus tuples which are sent to the committer operator. The committer validates each incoming consensus tuple by ensuring that two things are true: (1) the view number in the tuple is the one that matches what the committer is currently viewing and the (2) sequence number belongs to the committer's current sequence number window, that is proposals Must come without any breaks and gap and accepted sequence. When everything validates, request stores by the committer in its window and the consensus tuple re-emits as a commitment.

If a view tuple with a view change notification is received by the committer, then it updates the present view number of it. As soon as a new view is accepted, the committer gathers all the entries. from its in-memory record window—the latest view and request that it has seen for each sequence number - and emits them as a record tuple. The record tuples are sent on via the record sink to the record source and the new 'proposer' picks them up during view change recovery.

When the committer receives a view tuple signaling a view change (Section 4.4), it updates its current view number. Upon accepting a new view, the committer collects all entries from its in-memory record window - the most recent view and request it has seen for each sequence number - and emits them as a record tuple. These record tuples are forwarded

via the record sink to the record source, where the new proposer collects them during view change recovery.

4.2.4 Request Execution and Result Delivery

Consensus tuples from the committer are routed to the executor operator, which applies the committed requests to the application state machine. Each executor maintains a state, including an execution index, a deduplication filter tracking executed payloads, and a result cache that maps each client ID to its response.

Upon receiving a consensus tuple, the executor first checks that the view number matches the current view. It then accumulates arriving tuples until at least $F + 1$ committer instances have delivered the same tuple for the same sequence number, ensuring it only executes requests that have reached distributed agreement. Once this quorum is satisfied, the executor processes each payload in the request. It compares the transaction number of the payload against the largest transaction number stored for that client in the results cache: if the payload has already been executed, it returns the result that is already in the result cache; otherwise it calls the application logic, updates the deduplication vector and stores the response in the result cache.

Results are then sent to the reply sink operator, an output node that returns results to the originating client(s). Filtering, for example by deduplicating the information that is processed by the same executor, and caching the result is important since a crashed executor replica might reprocess information already processed correctly before the crash. Being able to return the same result to such duplication without re-executing the application logic is vital for correctness if the application is not idempotent - this is called caching.

When an executor receives a view tuple indicating a view change (Section 4.4), it updates its current view and resets its internal state accordingly, so that it only accepts commit quorums that belong to the new view.

4.3 Garbage Collection

Gumti runs a perpetual query process, and operators keep an internal state, including a window for requests, a sequence number of issued requests, and commit records. If allowed to persist and not cleaned properly this state can become limitless. Implement garbage collection to enable operators to safely remove the old state that they no longer require for recovery purposes from Gumti.

4.3.1 Flink Checkpoint for Durability

The executor reports/commits its execution progress after it has executed a checkpoint number of commands. This published progress is read into a dedicated GC Source which pushes the new GC Tuple, which has a current sequence-number milestone, into the dataflow. This will separate the GC signalling from the normal executor's processing path, and let all downstream operators get the pruning signal through the normal stream topology.

4.3.2 Progress Tracking and State Pruning

When a proposer or committer receives GC tuples, it waits until it has collected values from at least $F + 1$ different replicas. It then sets its stability threshold to the $(F + 1)$-th highest sequence number among those values. This ensures when maximum replica agree on the new value only then threshold advances. The GC Source monitors the progress of each executor. It chooses the $(F + 1)$-th highest reported sequence number as the new stability threshold - this guarantees that at least one copy of the corresponding checkpoint is always available. Whenever this threshold increases, all proposer, committer, and executor replicas are informed by the GC Source. For the proposer, this advances the lower bound of its sequence number window. For the committer, this moves its window forward and removes records for sequence numbers that are now below the stability threshold.

The executor also responds to incoming GC tuples by advancing its execution index, discarding already-processed entries and keeping memory consumption bounded. Operators at all stages retain only the recent state required to process future requests and recover from failures through the view-change record replay mechanism.

4.4 View Change Mechanism

In the event of the failure of the active proposer, the proposals in Gumti has to be elected with another proposer and processing shall continue with this proposer without loss of previous committed requests. In this section it is explained how this is done: in Section 4.4.1 leader failure detection is explained, in Section 4.4.2 the propagation of the change of view, in Section 4.4.3 state reconstruction from committer record, and in Section 4.4.4 how the new leader resumes processing.

4.4.1 Leader Liveness Monitoring via Timers

The controller operator monitors whether the active proposer is still making progress. To do this, it tracks two values: the highest request number that each request source has assigned, and the highest request number that executors have completed. Each request source periodically emits a `ProgressTuple` tuple with its highest assigned request number, which the controller records as the target and each executor periodically emits an `ProgressTuple` tuple with the highest completed request number for each source, recorded as the actual.

The controller verifies if the number of requests that have been completed is increasing at least $F + 1$ executor instances. As long as the controller does not detect any growing, despite receiving reports from the request source that there are new incoming requests, it assumes that the active proposer failed and that the consensus is stuck - even when the controller has a timeout (we use a timeout of 2,000 milliseconds in our current implementation). And then it increments its local view counter and emits a `VIEW` tuple to announce the new view number.

4.4.2 View Change Propagation through Streams

After the controller sends a `VIEW` tuple, it is sent to the view source which pushes it to all the proposer, committer and executor replicas. This is distributed on the same pattern according to stability threshold in Section 4.3.2. When the `VIEW` tuple arrives at each replica, it updates its current view number. At this point, no more tuples with the old view number will be taken by committers/executors.

All the partitions operate a view-change process individually. The following equation is used to determine the new active proposer for a partition: `p = v % (f+1)`, in which `v` is the new view number. This guarantees deterministic and consistent selection of a partition leader.

4.4.3 State Reconstruction from Durable Logs

To make sure that requests committed in previous views are not lost, each committer keeps a record `d = (s, v, req)` for every sequence number `s` in its window. This record stores the most recent view `v` for which the committer received a proposal and the request that came with it.

When a committer learns about the new view, it takes all its records, puts them into a set `D`, and emits a `RECORD` tuple to the record sink. The record sink then forwards these tuples through the record source to the new proposer.

The new proposer does not start working immediately. It waits until it has a set of `RECORD` tuples from at least $F + 1$ different committers. This is very important as this goes through a consensus session with confirmations from $F + 1$ committers. Thus if the request had previously gone through, then at least one of the supplied records will have the assignment of sequence-number for that request. Thus, there is no need to worry that a committed request will be lost.

Once the proposer has enough records, it goes through each sequence number covered by those records and picks the request with the highest view number as its new proposal for that sequence number.

4.4.4 Resuming under the New Leader

After the new proposer has looked through all the records collected from the committers by the new proposer, it knows what previous view recorded at least one of the committers' requests, if any. The protocol does not depend on exactly what was done, since some of the requests may not have been committed (because they were not registered by a quorum of committers before the view changed). If any of the other sequence numbers in the sequence are not associated with any record, the proposer may use that sequence number with a new incoming request. This is where the newly appointed leader assumes full control and starts receiving demands as usual.

Committers and executors also asynchronously change their internal view number. Once they have this update they will only process tuples that are part of the new view. If

there is any tuple coming in has the old view number – simply reject it – it will not be processed. This keeps any traffic from the former view from disrupting the mechanism of consensus formation.

Together with the record collection phase, this step for resumption secures a neat and secure transitioning of leadership from the old leader to the new one. It does not skip any previously committed request and continues moving forward from any place where the previous proposer left off.

4.5 Gumti Architecture and Flink Integration

The main contribution of Gumti is the mapping Tara replication protocol to Apache Flink. This section explains how this integration works: how we use Tara's node to Flink operators, how the replicas interact with the Flink runtime during execution, and which Flink services Gumti relies on to avoid building infrastructure from scratch.

4.5.1 From Tara Nodes to Flink Operators

Tara was originally specified as a directed acyclic graph (DAG) of processing nodes in which tuples flow from source nodes through processing stages to sink nodes. Gumti preserves this graph structure and maps each Tara node directly to a Flink operator. Figure 4 illustrates the resulting Flink job topology.

Each Tara node type is mapped to the Flink abstraction whose properties match its role. The source nodes (request, view, record, GC source) become `RichParallelSourceFunction`: as sources they ingest external input into the dataflow; being parallel, multiple instances can run concurrently (e.g. one request source per client-facing port); and the `Rich` lifecycle (`open()`, `RuntimeContext`) lets each instance configure itself per subtask — binding its port and registering in ZooKeeper using its subtask index. The processing nodes (proposer, committer, executor, controller) become `KeyedProcessFunctions`, because keying provides deterministic routing of each tuple to a specific instance — replacing Tara's `CustomStreamGrouping` — while `processElement()` and keyed timers give the per-record, state- and time-driven control the protocol logic requires. Sink nodes (reply, view, record, GC sink) become `RichSinkFunctions`: they can be used to perform the outbound side-effects that end the pipeline and each can set up and tear down its outbound resource using the `Rich` lifecycle.

4.5.2 How Replicas Interact with the Flink Runtime

In the case of Tara each of the stages of the protocol are re-replicated for fault tolerance. Replicas are actually realized by Flink as runs as parallel instances of the same Flink operator, known as operator parallelism in Flink. If an operator in Flink has been configured with a parallelism of n, then Flink will start n parallel subtasks (also known as operators) of it, each of which uses a TaskManager task slot independently. The state of each subtask is independent. whereas in a typical keyed Flink job each subtask would process its own distinct set of keys, Gumti will send a copy of every tuple containing a

Table 1: Flink services used by Gumti

Flink Service	**How Gumti Uses It**
`setParallelism`	Deploys $F+1$ or $2F+1$ replica instances of each operator stage without any manual process management
`keyBy`	Routes each tuple copy to a distinct replica by its target key; used to broadcast a tuple to all replicas of a stage (Section 4.6), replacing Tara's framework-level routing.
`KeyedProcessFunction`	Supports per-key stateful processing that has guaranteed single-threaded processing per key (no locks) so replica logic can be lock-free
`timerService`	Calls the controller to report if a progress stall has occurred and trigger view changes without a dedicated timer thread
`union`	Merges several input streams into a single input stream to an operator, used to combine requests, view changes, records, and GC signals into each processing node's input

key it receives to all copies of a single stage and all $2F + 1$ copies can agree on the key (how this is realized is described in Section 4.6).

Gumti exploits this mechanism directly: the proposer is deployed with $F + 1$ parallel instances, the committer with $2F + 1$, and so on. Each subtask determines its own replica identity at startup by reading its subtask index via `getRuntimeContext().getTaskInfo().getIndexOfThisSubtask()`. This index is used to assign the subtask's partition, its node role (e.g., whether it is the active proposer or a standby), and its REST server port.

4.5.3 Flink Services Used by Gumti

Table 1 summarises the specific Flink services that Gumti uses and how each one simplifies the protocol implementation.

Of these, the combination of `KeyedProcessFunction` and `keyBy` is the most important. The group should agree and see all the tuples that are part of a partition in every replica of the partition. Gumti does just that with `keyBy` on a precomputed `targetKey`: every tuple is duplicated once per replica, and every tuple is keyed in a way that ensures that it is delivered to a different instance (Section 4.6). This will solve the issue if Tara had to route the layers.

4.6 Flink Cluster Deployment and Operator Parallelism

Gumti is a single Flink streaming job that we call `Tara-on-Flink` and submit to a Flink cluster, where the *JobManager* schedules all operators and the *TaskManagers* execute these operators in parallel task slots.

In addition to the mechanisms introduced in Section 4.4, Gumti makes use of a number of other Flink mechanisms. KeyedProcessFunction is a stateful operator which guarantees all tuples with the same key are processed by the same parallel instance and maintains this with the help of Flink, which is what Gumti leverages to ensure that the correct replica receives all the tuples in the partition. The keyBy() transformation gives a KeyedProcessFunction a certain partition of the stream containing only those tuples where the key is equal to the canonical key of the partition function, where the key is drawn from the key selector. A `RichParallelSourceFunction` is a Flink source that will be executed in parallel and each parallel instance produces tuples to the stream independently. Last but not least an operator can register a callback at a later time (with Flink's ProcessingTimeTimer), and Gumti's controller will use it to detect progress-stalls.

Operator Parallelism: There are F number of parallel jobs that every operator of Gumti can deploy. The replication level of each operator is given in table 2.

The committers executor, controller and all source operators except the request source that has requested the setup run with $2F + 1$ instances. That way, when F instances fail to respond, the remaining $F+1$ instances will be able to make progress. The proposer will be using $F + 1$ instances and so will the request source, but for different reasons. $F + 1$ is the number of instances used by the proposer because exactly one proposer is active as per view change; other instances are simply replica instances that are ready to take over in case of a view change round. The request source requires $F+1$ replicas as it is not part of the consensus quorum, but only need to be reachable to clients. The clients can access any of the replicas and recover if one fails, so $2F + 1$ replicas will be too expensive, since a single replica will satisfy the requirement of clients to continue accepting commands.

Tuple Routing via keyBy: Each tuple carries one integer field, its `targetKey` and all the processing operators in Gumti (proposer, committer, executor and controller) are keyed on this integer field, via `keyBy(t -> t.targetKey)`. Flink ensures that all tuples with the same key end up in the same parallel instance of Flink, and that is what this field determines which replica will process a particular tuple.

Table 2: Operator replication levels in Gumti (conceptual design)

Operator	**Replication**
Request source	$F + 1$
View source	$2F + 1$
Record source	$2F + 1$
Controller	$2F + 1$
Proposer	$F + 1$
Committer	$2F + 1$
Executor	$2F + 1$
Garbage collection source	$2F + 1$

Gumti does not send a tuple to one replica; it sends a copy to all replica of the target operator. This is the reason these operators run at $2F+1$: each committer, executor, and controller has to see every tuple so that any $F+1$ of them can form a quorum and agree, even after F instances fail. Routing each tuple to a single instance would shard the work and break the replication that the quorum depends on.

This fan-out is done by a helper, `broadcastToSubtasks(stream, N)`, which creates `N` copies of one logical tuple, one per replica. Before being forwarded by `keyBy`, a copy of `s` is stamped with `targetKey = keyForSubtask(s, N)`.

`keyForSubtask` exists because a plain `s % N` does not actually reach instance `s` in Flink. Flink does not map a key straight to a subtask. It first hashes the key into one of a fixed number of key groups (128 by default) and then assigns whole key groups to subtasks, so consecutive integers do not fall on consecutive instances. With three committers, the keys 0, 1, and 2 all hash into key groups owned by the same instance, so using them directly would deliver all three copies to one committer and leave the other two idle. `keyForSubtask(s, N)` works around this by searching for a key whose hash lands on instance `s`.

The proposer and committer fan out their own output the same way. When the proposer issues a proposal it loops over the committers and emits one copy per committer with the matching `keyForSubtask` value, and the committer does the same toward the executors.

One bit of clarification about the executor, as its role can be misunderstood. It is a copy not a fragment. Each command pushed into an executor is pushed by every executor, preserving the $2F+1$ copies of the application state to be in step. partition is a field in a tuple to split commands across executors is never used and is used only to split per-partition state in an executor. When the `PARTITIONS` is 1, there is one partition, and deployed configuration, each executor run on the same command log.

Additional Configuration:

- Sequence number window size: 2000 entries for each operator window.
- Checkpoint interval (application-level): every 1000 committed commands — controls when executors emit CHECKPOINT tuples to the GC pipeline.
- Controller timeout: 6000 milliseconds. This was increased from a previous value of 2000ms: at 2000ms, then the controller by that time, has to switch another view to be taken, before another recovery of the previous controller takes place, leading to view change cycles without resuming (applying for some time now, 6000ms remains ample time for a view change to be completed).
- Batch size / timeout: up to 200 payloads per batch , with a 1000 ms timeout before a partial batch is emitted (Table 3).
- Service discovery: *ZooKeeper* is used to publish and look up the TCP addresses of replicas of the request source, to which the client can connect.

Table 3: Batching parameters in request source

Parameter	Value
Maximum batch size	200 payloads
Batch timeout	1000 milliseconds
Parallel instances	2
TCP port	11100

Table 4: Mapping of Tara nodes to Flink implementation components

Tara Node	Flink Class	Flink Operator Type
Request Source	`FlinkRequestSource`	`RichParallelSourceFunction`
Proposer	`FlinkProposer`	`KeyedProcessFunction`
Committer	`FlinkCommitter`	`KeyedProcessFunction`
Executor	`FlinkExecutor`	`KeyedProcessFunction`
Reply Sink	`FlinkReplySink`	`RichSinkFunction`
Controller	`FlinkController`	`KeyedProcessFunction`
View Sink	`FlinkViewSink`	`RichSinkFunction`
View Source	`FlinkViewSource`	`RichParallelSourceFunction`
Record Sink	`FlinkRecordSink`	`RichSinkFunction`
Record Source	`FlinkRecordSource`	`RichParallelSourceFunction`
GC Sink	`FlinkGCSink`	`RichSinkFunction`
GC Source	`FlinkGCSource`	`RichParallelSourceFunction`

4.7 Core Data Types and Communication Primitives

Table 4 presents the mapping between Tara nodes and Flink classes, highlighting the specific Flink functions associated with each Tara node.

Any communication between operators of Gumti is by typed tuple system. All the types of tuples are subclasses of the java abstract base class BaseTuple, which all the messages in the system must be subclasses of. It has four fields which are common: NodeId of the sender (node type and numeric node id), a partition number, which shows in which consensus partition this tuple belongs, a targetKey, which carries the identifier which can be used to get the tuple to a specific operator instance and a tag called TupleType. The TupleType is inherits from Tara and is used for driving stream grouping by Heron. The Flink port routes by Java type instead (through `instanceof` checks), so this field is kept for parity, but is not used for routing.

The seven concrete tuple types are used to represent the communication in the protocol:

RequestTuple is a class that contains a `Request` from a RequestSource to the proposer. Wraps the incoming client command and broadcasts it to each proposer replica via its `targetKey` such that the active proposer can order it, and the standby proposers are able to buffer the command in case they become the next leader due to a view change.

ConsensusTuple is emitted by the proposer and contains the four fields of a PROPOSE message: `partition`, sequence number `snr`, `view`, and the corresponding `request`. This tuple is sent to the committer who uses it to store the sequence-number assignment.

ViewTuple carries a new view number and partition from the controller to all proposer,

committer, executor, and controller replicas. It is delivered to every replica instance through the `broadcastToSubtasks()` mechanism, which sets a different `targetKey` on each copy.

GCTuple is emitted by the executor after a checkpoint. It carries a checkpoint number `cnr` and a single-element set holding the id of the executor that produced it. The `snr()` helper method derives the corresponding sequence number from cnr using the checkpoint interval. Like ViewTuple, it is broadcast to all proposer, committer, and executor instances. A recipient treats a checkpoint as stable once it has received matching GCTuples from $F+1$ distinct executors.

RecordTuple is emitted by committers during a view change. It carries the `partition`, the current `view`, and a `RecordWindow` object containing all sequence-number records the committer holds. The new proposer collects these tuples from $F+1$ committers before resuming.

ReplyTuple is emitted by the executor after command execution. It carries the `partition` and a `Payload` result object, which is forwarded through the reply sink back to the client.

ProgressTuple provides a way to report progress to the controller by both the request sources and the executors. It has an embedded `progress` array of size `REQUEST_SOURCES` with each element containing the highest request number from the associated source. Request sources that they report the number of request sources they have assigned, and executors that they report the number of request sources they have completed.

4.8 Key Adaptations from Original Tara Design

While Gumti follows the core protocol logic of Tara, several adaptations were necessary to implement it on Apache Flink.

broadcastToSubtasks(): The operations of tuple routing are delegated to the custom stream groupings that are available in Heron. The `keyBy` operator is used to send a tuple to a single instance and it is however not built into Flink to send a tuple to all instances of a keyed operator. Gumti circumvents this by implementing a function, `broadcastToSubtasks()` in App.java. Emit one for each replica for every tuple that must be replicated (a request, a view, a record or a GC tuple, depending on the operator), and for each replica, set its `targetKey = keyForSubtask(s, N)`. Downstream operator's `(t -> t.targetKey)`.Then distributes the copies to each instance if they are specified in targetKey. No plain targetKey % N since Flink hashes a key into key groups before passing them to instances, and since numbers passed consecutively do not mean that Flink instances will be accessed sequentially, so keyForSubtask is passed to find the hash value of the key for the appropriate instance.

Timer-Based View Change Detection: By comparing the target and actual progress counts, controller detects a stalled leader but does not specify a particular timer mechanism. In Gumti, the `FlinkController` uses Flink's `ProcessingTimeTimer` - registered via `ctx.timerService()` - to periodically check a computed deadlineMs. The deadline is recalculated each time progress is updated: if `actual[i] < target[i]` and no progress has been seen within `CONTROLLER_TIMEOUT` (6000 ms), the controller triggers a view change. This is an implementation of TARA's liveness monitoring concept on Flink's own timer

infrastructure.

Tuple fields promoted onto BaseTuple, plus a `targetKey`: In Tara, the tuple's `partition` and type are stream fields that appear at the beginning of the tuple in the groupings; `BaseTuple` only contains the sending id. That is because Flink's `keyBy` cannot take a positional field that is consumed in a grouping, but it requires an explicit key value, which is why `partition` and type are fields of `BaseTuple` and a field `targetKey` is added.

Controller timeout increased from 2 s to 6 s: Tara sets her controller timeout to 2000 ms. We saw a recovery round after a leader failure (collecting committed records over the HTTP feedback loop, replaying, and resuming) did not run to completion (in 2000 ms) on one node, and the controller re-triggered before recovery got back on track and the system stabilized. Increasing it to 6000, allows for a view change to be completed.

Type-dispatched routing moved into the dataflow: In Tara, Heron's stream groupings do the routing, and they also filter by tuple type. The proposer's grouping, for example, only lets request tuples through. Flink's `keyBy` cannot do this: it can send a tuple to a specific instance, but it cannot pick which tuple types an operator receives. So Gumti adds that step itself. Before each `keyBy`, a filter operator keeps only the tuple type that operator needs, using a simple `instanceof` check. The committer's output, for instance, is split into record tuples and consensus tuples, which are then processed separately. What Tara handles inside its routing, Gumti handles with plain filter operators in the dataflow.

4.9 Technical Challenges and Engineering Solutions

Porting Tara to Flink surfaced several problems. The routing and timer mechanisms behind these fixes were described in Section 4.6 and 4.8, this section covers the problems those mechanisms had to solve.

The view-change record loop silently dropped every message. After a leader failure the new proposer rebuilds its log from records sent by the committers. In early runs this never completed: the proposer waited for records that never arrived, and the system kept electing new leaders without progress. Two faults were responsible. The record sink posted to the path `/recordSource` while the record source listened on `/source`, so every request returned 404 and was discarded. Once the paths were aligned the source returned HTTP 500 instead, because the JSON body was deserialized straight into the concrete `RecordTuple` class, which Jackson cannot do while `BaseTuple` carries the polymorphic `@JsonTypeInfo` annotation. Neither failure raised an error on the sender side, which is why the loop looked healthy while delivering nothing. Aligning the two paths and deserializing through `BaseTuple` before casting fixed it.

Failover did not stabilize. Even with records flowing, a failed leader did not recover cleanly, for two reasons rooted in the controller's timing. Under steady load it never triggered a view change, because the stall timer was reset on every incoming request rather than when execution advanced, so a busy client hid a dead leader. Once that was corrected it swung the other way and re-elected a leader every few hundred milliseconds, before any of them could recover, which drove the view number up without restoring throughput. Tying the timer to execution progress, giving each newly elected leader a grace window before the next escalation, and clearing a pending view change only after real

progress is observed, removed both behaviours. A specific fault that sat in the proposer – when the garbage collection had not ran yet, the proposer propose from sequence number zero - and committers said ”no, that sequence number is already committed”. It now regenerates its log based on committers' records when it streams the view change. This controller timeout was also increased in the same fix (Section 4.8)).

Recovery ran out of memory under load. When using failover with low load the job crashed during a view change with an out of memory error at a higher request rate. Recovery requires a fixed window of two thousand record slots per record tuple and a stream of these during recovery and a blocked client session that continues to push data (an open-loop client) into a blocked system exhausted the default task-manager heap. The failover experiments thus involve the use of a closed loop client, that does not pile up of work in whatever manner during the outage, but stops sending while waiting for the replies, and the task-manager heap was increased to 8 GB. The oversized record window is left as future work.

Null requests took more time than larger payloads. The first batch of tests had some unexpected results, as requests null payload took longer than 5 kilobyte requests. It was not because of the protocol but how Flink moves data between operators. The records are collected in a network buffer; a flush timeout occurs if the buffer is not shipped after 60 seconds. Empty request never fills a buffer, so each wait results in a full timeout and consequently the cost of each wait is multiplied over the pipeline to significantly exceed the cost of the actual processing. More data = faster because it is flushed early, occupying a lot of buffers in the process. Experiments varying both the buffer and the flush timeout parameter showed this: After all, the empty requests generate very little traffic compared to what the smallest buffer Flink allows them, and the latency is almost linear in the flush timeout. Thus, the assessment will make the buffer timeout equal to zero, meaning that all records will be sent immediately. While the documentation seems to recommend not setting a timeout to 0, that recommendation is mainly targeted towards pipelines transporting large amounts of records; for Gumti's message rates, a 0 timeout was not an issue. The measurements in support of these observations are given in Section 5.

4.10 Useful Flink Features but Not Used

Broadcast streams: Flink also has a built-in way to send a message to everyone: a broadcast stream is copied to every instance of an operator, and each instance keeps it in a special kind of state. This looks like a perfect match for Tara's one-to-all messages, and it is the natural sibling of the keyed routing Gumti uses. It was not used, because the receiving operators have to do their normal keyed processing as well, and the operators are noticeably more complex if a broadcast input is mixed with keyed state. One keyed copy per replica provides the same delivery by one consistent delivery mechanism.

Managed state and state backends: Flink can also manage an operator's state for you: the state is registered with the framework, saved automatically in every checkpoint, and can be stored in a backend such as RocksDB, which keeps it on disk so it can grow larger than memory. Gumti does not use this and keeps its protocol state in plain Java fields instead, for two reasons.

The first concerns recovery after a leader failure. Now the new leader will restore its state from the committers' records since this is prescribed by the protocol; a restored state from an old Flink checkpoint might be stale because it is missing what the other replicas have committed in the intervening time. Managed state simply is not a mechanism utilized by this protocol. The second is more interesting as Gumti does take its own checkpoints. When garbage collection happens, each executor updates the disk with its current state at regular intervals of the sequence of requests and lets the rest of the executors know how much it has completed. All Replicas agree on a safe point and drop all state that lies below the agreed safe point. The snapshots are indeed protocol data, produced at an arbitrary point in time by the protocol, broadcasted to the replicas, read by each executor to catch up with the protocol and discarded. This cannot be done in the managed state of Flink. Its checkpoints only occur when the system operates and are out of sight within the framework, and are activated only upon the restart of the entire job. It would be just another uncoordinated snapshot mechanism beside the protocol's one that would turn this on. This is prevented by keeping the snapshots in the protocol's hands and will make recovery engine independent.

Event-time processing: Gumti uses Flink's processing-time timers, but most of Flink's timer machinery is built for event time: records carry timestamps, watermarks track how far the stream has progressed, and timers fire based on that progress. This is the right tool when the data itself has a timeline, for example sensor readings arriving out of order. Gumti's messages do not have such a timeline - only the order of processing is important - and the failure detection must follow the real clock. Event time machinery was thus never utilized, and the simpler processing-time variant was the right one to choose.

4.11 Summary of Design Decisions

This section outlines the major architectural decisions in Gumti and the reasons for adopting them.

Using separate nodes for different tasks instead of a single monolithic replica: Gumti is based on the directed acyclic graph model similar to that proposed by Tara, of small, single-responsibility operators: proposer, committer, executor and controller, and source/sink nodes for replication. Operators are only provided with the state required. This helps in making individual stages easier to test, reason about, and debug than a monolithic replica which would combine all of the protocol logic into a single unit.

Flink's `keyBy` for replica routing instead of a dedicated router: Gumti forwards tuples through keyBy mechanism provided by Flink, instead of creating a separate routing layer. However, all the behaviour required by the protocol with respect to broadcasting to the replicas is provided as a layer on top of keyBy (Section 4.6). This prevents routing to be put in custom infrastructure code.

REST over HTTP for feedback paths (Sink → Source): The dataflow model in Flink only supports tuples to flow downstream. For paths that need to feed information back upstream - such as view-change signals, garbage-collection thresholds, and record recovery - Gumti uses REST over HTTP. Every source operator exposes a light-weight http endpoint, and the related sink posts tuples directly to the endpoint. This is respectful of

Flink's execution model, but does not necessitate any changes in the framework.

Sliding windows for bounded memory: Gumti keeps the state of the protocol within a fixed-size sequence number window. When older data becomes stable, the window moves forward and outdated state is removed. This avoids memory usage from growing continuously over time. The approach is close to Tara's stability-threshold mechanism and helps keep ongoing deployments memory efficient.

Kryo with `Externalizable` for tuple serialization: Tuples cross two boundaries, and each uses a different encoder. In the Flink dataflow they are serialized using Kryo, which Gumti makes mandatory using `PipelineOptions.FORCE_KRYO` as the tuple classes were not plain POJOs, and therefore would have been serialized with a slower serializer. On the REST feedback paths they are encoded as JSON by Jackson; `BaseTuple`'s `@JsonTypeInfo` writes the concrete class name into a `_type` field, which is what lets the receiver rebuild the right subclass (ViewTuple, GCTuple, RecordTuple, and so on). The `Externalizable` methods and the `TupleType` field that the tuples also carry are inherited from Tara and are not on the active serialization path in the Flink port.

5 Experimental Assessment

The previous chapters showed how Tara can be implemented on Flink; this chapter examines how well the result actually works. Gumti is evaluated in three ways: whether it keeps running when a leader fails, and what throughput and latency it reaches with batched and with non-batched consensus. The experiments also show just how much of a factor the engine itself is in the outcome, and it is turned out to be one of the more interesting findings of this work. This chapter presents the objectives, the set-up, the measurements, and a discussion of the meaning.

5.1 Evaluation Objectives

The four questions are answered in the evaluation.

How do throughput and latency looks like in batched consensus? The first question is what performance Gumti will achieve in the usual case – using up to five requests in a single consensus instance. The load is incremented step-wise from having 1 client up to about 128 clients and for each step tested the throughput and latency. To see how the size of the data affects the system, this is done for four payload combinations: null request and null reply, null request and 5 KB reply, 5 KB request and null reply, and 5 KB on both sides.

How do throughput and latency looks like in non-batched consensus? The second question repeats the same measurements with non-batching, so that every request runs through its own consensus instance.

How much engine itself effects the results? The third question looks below the protocol, at Apache Flink. Gumti's messages travel through Flink's network buffers, and two of their parameters, one is buffer size and another one is the buffer flush timeout, and these two has a strong influence on the results. The assessment is conducted in both settings, in that some experiments are set up to isolate the protocol costs, and to justify the configuration used for the other measurements.

Does Gumti able to tolerate leader failure? A crash of a replica must not cause the system to crash. The desired behavior is tested by killing the active proposer twice while running the system and measuring its throughput over time, to see if the system detects the failure and elects a new proposer and then performs at normal speed.

5.2 Experimental Setup and Environment

All experiments are executed on a single machine (MacBook Air with an Apple M-series CPU and 16 GB RAM) with one ZooKeeper as discovery service, one TaskManager, eight task slots, and 8 GB heap. Gumti runs with $F = 1$, and the replicas occupy separate task

slots rather than separate machines. We use the buffer timeout is 0ms and 2ms, the buffer size is 128 KB, and batched consensus uses a batch size of five with a 4 ms batch timeout. Load comes from clients, which send a new request only after receiving the previous reply; their number is raised step by step from 1 to around 128. Each measurement point runs for 120 seconds with throughput and latency sampled every second. The first 30 seconds are discarded because the system is still warming up: connections are being established, the Java runtime is still optimizing the code, and the queues have not yet reached their steady level. The last 10 seconds are discarded as well, because the clients shut down during this phase and the final samples would measure a partially stopped system. The remaining 80 samples reflect stable steady-state behaviour; they are averaged, and every point is measured three times and averaged again.

5.3 Development Complexity and Code Characteristics

Without the benchmark clients, it consists of about 2,800 lines of Java code, not counting comments and blank lines, spread over 37 classes. The consensus logic makes up only a small part of this code. Most of the code connects the protocol to Flink: it closes the feedback loops outside the dataflow, runs the discovery service, and computes the routing keys. The protocol logic itself follows the structure of Tara and did not need much new code.

5.4 Performance: Throughput, Latency

This section presents the throughput-latency behaviour of Gumti, first with batched and then with non-batched consensus. For each of these configurations two measurements are taken, one with a buffer timeout of 0 ms and one with 2 ms, because the picture changes considerably in just one configuration of the single engine parameter.

5.4.1 Batched Consensus

Running with a buffer timeout of 0, the four combinations of possible payloads are illustrated in Figure 5. All curves look alike: Latencies are low in the beginning but increase as load increases, and when the workload hits an optimal limit, after which the curve turns up but rises rapidly with the number of additional clients. The null request/reply combination can connect to Gumti at 5,300 requests per second with a latency of around 3 ms, while the combination of 5 KB rep + 5 KB req reduce the capacity to 3,900 requests per second, and the mixed combinations give values between the two extremes. The latency ordering is the same as the payload ordering: null combination is the fastest for all loads, while the 5 KB/5 KB combination is the slowest for all loads. The single-client point sits apart from the rest at about 6 ms, because one closed-loop client can never fill a batch of five, so every request waits for the batch timeout before it is proposed. For this reason the batch timeout was lowered from its earlier value of 10 ms to 4 ms; this halves the single-client latency and has no effect on any other point, since from 16 clients on the batches fill by count.

Figure 6 shows the same experiment with a buffer timeout of 2 ms. Two changes are

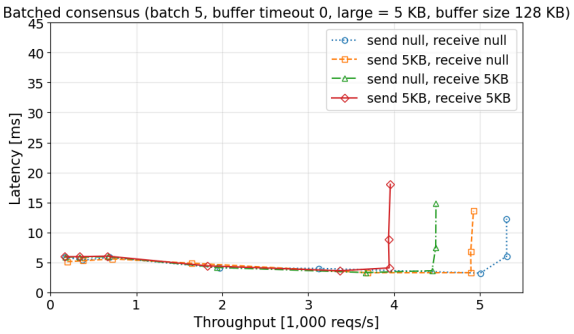


Figure 5: Batched consensus with buffer timeout 0

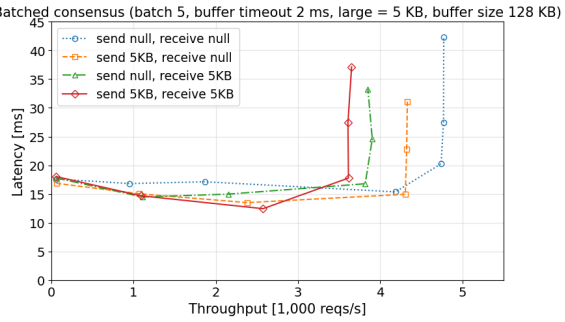


Figure 6: Batched consensus with buffer timeout 2ms

visible. First, all latencies are much higher: the floor moves from about 3 ms to 13–17 ms, because every message now waits up to 2 ms in the network buffer at each operator hop, and these waits add up across the pipeline. Second, the clean ordering is gone at light load: the 5 KB combinations now show lower latency than the empty ones, because their data fills the buffers and gets flushed early, while empty requests always wait for the full timeout. The buffer size plays no role in this: an empty request cannot fill a buffer of any size, so its latency follows the timeout alone, and only the large payloads react to how big the buffer is. The mechanism behind these effects is summarized in Section 5.4.3.

5.4.2 Non-Batched Consensus

The following two figures (Figures 7 and 8) repeat both of the above measurements, but use non-batching – each request is run through its own consensus instance. The curves are quite similar to the batched ones with a slightly reduced level—the empty combination achieves 4,900 req/s, and the 5 KB / 5 KB combination 3,600 req/s. The single-client point is different, however: if you don't have a batch to wait for, the single request makes it through the pipeline in about 2.5ms. The same two effects occur as in the batched case: the latency floor increases by 2-3 ms to 13-16 ms and the capacity decreases significantly, and even more in the case of the 5 KB combination. The loss is greater than for batching, because here, the loss of each buffer is due to a single request: there is no batching to speak of.

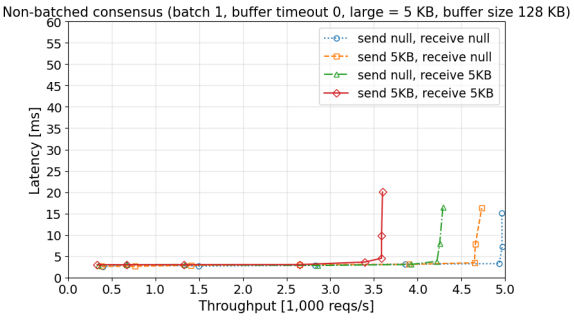


Figure 7: Non-batched consensus with buffer timeout 0

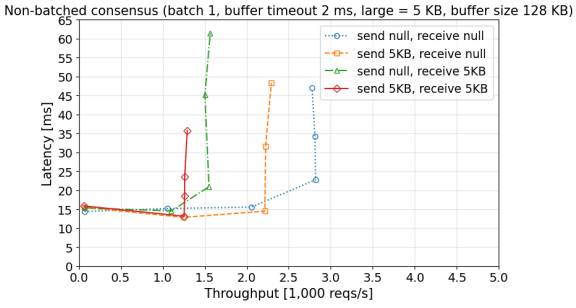


Figure 8: Non-batched consensus with buffer timeout 2ms

5.4.3 Influence of the Engine's Buffering

Flink forwards records between operators through network buffers, and a buffer is sent when it is either full or a flush timeout expires. This simple rule explains all the differences above. Empty requests never fill a buffer, so at each hop they wait for the timeout so that their latency goes with the timeout but does not care about the buffer size. Only large requests respond to buffer size because they fill buffers and get flushed early, and they could be faster as empty requests when timeout is high. Separate control measurements showed both relate: Latency of empty-payload operation is nearly linear in the timeout and is identical for all buffer sizes (from 4KB, the minimum allowed by Flink) up to 128KB. To make all primary results easier to compare with one another a timeout of 0 was used, with every record being sent directly and the measurements referring to the protocol and not the engine.

5.5 Fault Tolerance and Recovery Experiments

The last experiment tests the central promise of a replication protocol: the system must survive the failure of its leader. We test it with 4,000 requests per second for 180 seconds. The active proposer is killed at $t = 60\,\text{s}$; the killed replica is revived in some seconds later, and the then-active proposer is killed a second time at $t = 120\,\text{s}$. The kill is preventing the proposer from proposing new proposals, and that is the failure that the protocol should detect and react to: the controllers notice that execution is no longer progressing, but new proposals are arriving and they switch the role of leader and start processing the requests from the other proposer. The experiment was repeated three times, with the same result, as can be seen in Figure 9.

The figure tells a clear story. Before the first failure, Gumti delivers the offered around 4,000 requests per second. Upon the first kill of the proposer, ordering stops immediately: only the few requests that have been ordered may successfully finish; throughput drops back to zero. The kill time shown in the figure is one to two seconds after the kill time measured by the load generator; it is an alignment artifact between the load generator's measurement clock and the kill script not a delay in the protocol. The system then stays at zero until the failure is detected. Once the view change fires, the remaining steps are

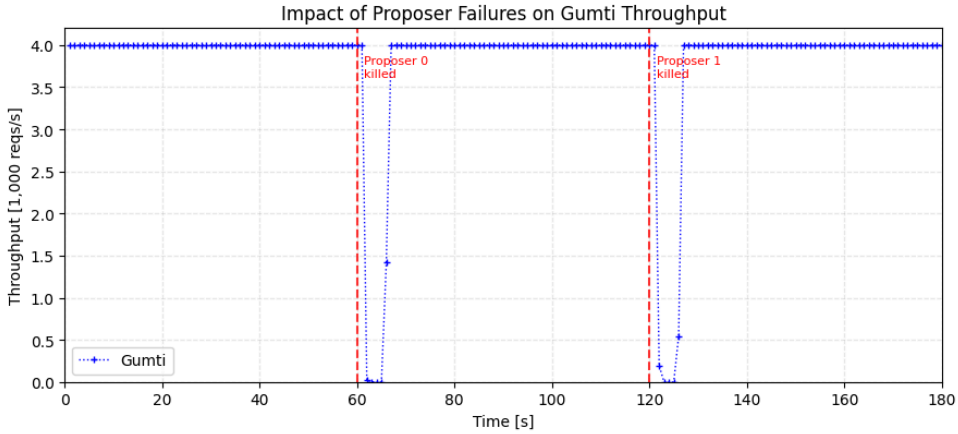


Figure 9: Gumti's fault tolerance

fast: the new leader fetches the committers' records, reconstructs the state of its records, and handles the queued requests. Overall, the system can re-establish full "rate" six or seven seconds after the kill - this is the detection timeout and a sub-second view change. The second killing at $t = 120\,\text{s}$ is repeated in the opposite direction to the first, so as to demonstrate that recovery takes place in both directions.

So, to sum up, Gumti has the same tolerance of the failure of its leader as the requirements demand: it takes a couple of seconds for the crash to halt the system and a new leader takes over, the system picks up the pace again. This is similar to what the original Tara paper shows on Heron, and shows that the fault tolerance mechanisms were carried over to Flink.

5.6 Discussion of Experimental Findings

The four questions from Section 5.1 can now be answered.

Questions one and two were about "throughput" and "latency. At the latency floor of 3 ms, with batched consensus Gumti serves nearly 5,300 requests per second with no payloads and nearly 3,900 requests per second with 5 KB payloads on each side, respectively. Without batching, those numbers are slightly lower, at nearly 4,900 and 3,600 requests per second. The comparison with the 2 ms buffer timeout explains why. There, consensus round costs are quite high (any message could be in the network buffers) and throughput about doubles when you batch five of them onto one consensus round.

This indicates the general evaluation lesson: performance a protocol can display on a general purpose stream engine depends on the configuration of the stream engine as well as the protocol itself. Buffer timeout had the effect of raising and lowering the latency floor between 3and 17, while flipping the impact of the ordering of the payloads at a light load and changing the impact of batching. A batch timeout had the same type of impact one level up, at loads too light for a batch. All of these effects are from 'send when full' or 'send when a timer fires' rules in the layers below her. To obtain trustworthy measurements it was necessary to identify these mechanisms and set them out of the way.

The final question was whether Gumti survives the leader failures, and the answer is yes. In every run, the system can detect the dead proposer, choose a new leader, and returned to the normal speed of 4,000 requests per second within the detection timeout plus a sub-second view change. The failover affects individual requests in two ways. Requests that were already inside the pipeline when the leader died came through after the recovery. Requests that arrived while there was no leader got no reply in our measurement. This is expected: in Tara, it is the client's job to send a command again when no answer comes back, and the executors throw away the duplicates. The recovery worked repeatedly within the same run and in both directions, so the fault-tolerance mechanisms clearly survived the move to Flink.

The experiments demonstrate that Gumti not only is a working reimplementation, capable of ordering requests at about the same rate as similarly configured on the original platform, but also remains robust and fault-tolerant with respect to leader failures by recovering quickly – in seconds – after each such failure and behaves as predicted by the protocol once the buffering of the engine itself is accounted for.

6 Related Work

The creation of reliable distributed systems is nothing new. Researchers and engineers for decades have been attempting to come up with various ways to create fault-tolerant and consistent systems. We included in this section some important papers that influenced our thinking about building Gumti. There was something important in each of the papers and together they reveal such a road that is pointing in the direction of what we are attempting to do.

6.1 Consensus and Byzantine Fault Tolerance

The first important paper comes from Castro and Liskov in 1999 [4]. They introduced something called Practical Byzantine Fault Tolerance (PBFT). The basic idea was deceptively simple but profound: what if some of the replicas in your system are not just broken, but actively working against you? What if they send wrong messages on purpose, or they are captured by adversaries? PBFT showed that a group of replicas could still reach agreement and produce correct results, even if up to one-third of them behave badly or maliciously. To achieve this, PBFT uses multiple rounds of messages between all replicas. In each round, replicas exchange their views about what decision should be made. Each replica checks the messages from others, and if most of them agree on the same value, then a decision is made with guaranteed correctness. This process is careful and complex, involving multiple phases and view changes, but it guarantees that wrong replicas cannot trick the system into accepting bad data or committing incorrect results.

PBFT shares several important principles with Gumti. Both systems face the challenge of maintaining agreement across multiple replicas despite failures. Both must handle the increasing complexity that comes with having more replicas: more replicas means more communication (in PBFT through explicit messages between all replicas, and in Gumti through the distributed consensus stages in the stream processor). Both must ensure that the system makes progress even when some components fail [4].

6.2 Stream Processing Systems

With increasing demands for volume of data, researchers started considering processing massive amounts of data as continuous streams [10]. Many applications required real-time information: the "batch" method of gathering data, putting it into a database, and then using "batch" jobs to process the data were too slow. For companies it was important to get real-time analytics, such as the trending topics, or figuring out real-time when there was something that was not right. A significant system to tackle this was S4 (Simple Scalable Streaming System) from Yahoo by Neumeyer et al. The main goal of S4 was to handle the enormous amount of data flowing continuously from a rich variety of sources, in some cases millions of events per second. What was clever about S4 is the way it was organised. Rather than performing all the interactions in a centralized machine (which would be very slow), S4 distributed the interactions among a collection of machines. It did it with a technique known as key-based partitioning: data which shares the same key always flows to the same machine. For example, if you were tracking events from many

users in real time, all events from user A would go to machine 1, all events from user B would go to machine 2, and all events from user C would go to machine 3, and so on. This way, each machine would not have to communicate with other machines for each and every event, but would be able to communicate for updates of individual events, e.g. how many events each user had sent, etc. S4 demonstrated this partitioning method to be effective and to be potentially scaled up to manage an extremely large volume of data being streamed in.

In Gumti we have a similar partitioning concept, but it is applied to consensus. In addition to consensus using Tara's approach, we also breakup the work to multiple processing engines. To send requests to the different states of processing depending on their location in the consensus process. Proposing (assigning a sequence number to a request) is one logical partition, committing (confirming the sequence numbers agreed upon by all replicas) is another partition and handling the request is another partition. While S4 divides regular stream processing workload between multiple machines in the cluster, it can also divide consensus workload between different machines in the cluster. Both S4 and Gumti utilize it for analytics and real-time aggregation and implementing consensus, respectively. This demonstrates that this is not a domain issue for analytics, it is a scaling distributed systems issue.

A key system added by Zaharia and co-authors [15] around 2013 is Spark streaming. One of the major differences in the architecture of S4 versus Spark Streaming was in how stream processing was being done. And while S4 continuously processes events one-by-one, Spark Streaming adopts an alternative approach: it batch processes a small number of events that show up within a window of a certain interval of time. It could gather events for 500 milliseconds or a second and then process them all at once together, collecting the next set of events, and so forth. A good side effect of this micro-batch approach is that, failure (or batching crop) will result in same output every time since each micro-batch is generated by same data with same code. The computing is deterministic, so if a machine crashes during the middle of a batch, the batch can be replayed from the stored input and the machine will get the same results. Deterministic replay was proposed to Spark Streaming as a way of offering very strong guarantees: when reprocessing the same batch twice, you always get the same answer. This is significant since you don't need to adopt costly co-ordination protocols to maintain replica consistency; replicas simply read the same sequence of data, and determinism assures that they get the same result.

Gumti is built on the Spark Streaming's principle of deterministic execution. A consensus protocol in our system "total ordering" makes sure that all replicas process requests in the same order. Afterwards, as the execution of the state machine is deterministic, the results of all replicas will be identical [9]. When a replica fails, and then recovers, it will be able to replay the decisions and requests to the consensus and again get to the same state as before the failure. Both Spark Streaming and Gumti rely upon the principle of determinism which means if there is the same input and same code, then there must be the same output. It is important to observe, however, that the Gumti consensus protocol also assumes ample communication between replicas. During normal operation, the consensus algorithm has to be executed on each decision to ensure agreement, such consensus communication [9].

After a couple years, Google has added something called the Dataflow Model [1]. An important problem the Dataflow Model attempted to solve was: What if I want to write

code in such a way that it can process batch data - where I can read all the data in one place and then work with it all in one place and stream data - where I receive all the data "one at a time" and work with it "one at a time"? These two were traditionally very different. With batch systems, such as MapReduce, you had to write code that handled the notion of full datasets and files. Stream systems required you to think about state management and handling late-arriving data. The solution offered by the Dataflow Model was to provide a single, unified abstraction which is flexible enough to work for both. In the Dataflow Model, you describe what computation you want to do (the "what"), and the system handles whether that computation happens on a batch of data or on a continuous stream (the "where"). The model also continues to deal with real-life problems, such as: what if events are received in an "out of order" situation, and you want to compute a sum or average? When should we produce results immediately, or after waiting a bit to ensure we have received all late-arriving data? To deal with these questions, concepts such as watermarks were introduced with the Dataflow Model. Watermarks are a way of saying "we have received all data up to time T, so we can produce intermediate results now". This way, stream processors can deal with both low latency results and correctness.

Apache Flink, the stream processor we use for Gumti, was influenced by many of the ideas from the Dataflow Model, especially its unified vision of batch and stream processing. By building Gumti on top of Flink, we can use all of Flink's rich features for handling time semantics and producing results at different levels of consistency. Unlike some other approaches to replication that require replacing the entire stream processor or fundamentally changing how it works, Gumti shows that strong consistency from consensus can be integrated with the sophisticated features that modern stream processors provide. For example, Flink's concepts of event time, processing time, and watermarks still apply in Gumti.

Recently, lightweight asynchronous snapshots for distributed dataflows have been the focus of research by Carbone and collaborators on the Flink project [2]. Their work focuses on the problem of capturing the "state" of a distributed stream processing system efficiently, without blocking the computation. Our implementation of Gumti is built on Flink and shares its concepts of snapshotting for its recovery, which makes this recent research in efficient snapshot execution relevant to our implementation. Fault tolerance and state management are key areas that modern stream processors are still continuing to develop and enhance, both of which are essential to implementations such as Gumti.

6.3 Data Flow as a Unifying Concept

Finally, we return to an older system which had a significant influence in shaping distributed computing: Dryad, an system created by Isard and colleagues at Microsoft in 2007 [5]. Dryad introduced a fundamental way of thinking about distributed computation that became important in the field. The concept was simple: You write a graph of the stages of your computation, and channels convey data from one stage to the next. Each stage of the process reads data from the input channels, processes the data, and writes the results to output channels. So this graph is automatically passed to the Dryad system which maps these stages to machines in a cluster, and gives you all the networking - you do not write any yourself. If a stage fails or produces corrupted output, Dryad automatically re-executes it to recover from the failure. This way of thinking computation

as a directed acyclic graph of data-flow stages became a central concept in distributed computing. The concept was elegant as you were able to distinguish your computation (what you were computing in each stage) from distribution issues (how do you manage to distribute stages on machines, how do you manage failures, how do you pass data). Similar ideas are used in Spark for batch processing and in Flink for stream processing, showing how powerful and general this abstraction is. Almost all of today's distributed data processing systems are based on the idea behind Dryad.

Gumti takes Dryad's vision and extends it in a new and important direction. Dryad showed that you can represent any distributed computation as data flowing through stages in a directed acyclic graph. This idea was adopted in MapReduce or Spark: various stages may be loading files, filtering them, grouping, and etc., and output the result. Flink uses this idea for stream computation, different stages might read from Kafka, process events, aggregate them, and send results downstream. In particular, we demonstrate that even the complex distributed algorithms, such as consensus can be represented as stages in a data flow graph when implementing them on top of Flink. This indicates that the data-flow abstraction is even more general and more powerful than it has been thought of. Not only for analytics or batch processing but it is flexible and expressive for sophisticated coordination or agreement problems. This gives the opportunity of implementing many other distributed algorithms (election, aggregation, indexing etc.) based on this abstraction.

6.4 Positioning Gumti in the Research Landscape

All of the above papers are pieces of important work in their respective fields. They evolved somewhat separately though: consensus research led to correctness and fault tolerance, and stream processing research led to scalability and then low latency processing. What Gumti has done, is to deliver the message that these two areas can be tied up and work very well. We show that the stream processing model offers both correctness guarantee for consensus-based replication and the scalability productivity of the modern stream processor through the use of a consensus protocol. This work has indicated that other similar stream processing abstractions could be used to possibly design other algorithms and coordination means in future systems.

7 Conclusion

Tara showed that a replication protocol can be built as a stream-processing application. So far, this idea had been tested on just one engine, Apache Heron. This thesis asked whether the same approach also works on a very different engine. To find out, we designed and evaluated Gumti, a reimplementation of Tara on Apache Flink that keeps Tara's structure and its correctness and fault-tolerance guarantees. The work illustrates that there is some cross over in the approach. Most of Tara's design maps directly onto Flink, the few things that Flink does not support are done outside the dataflow, and the resulting design is the same as before when dealing with leader failures. There is yet another twist being revealed: in this context, one setting for the engine level can make as much a difference as the protocol itself. This chapter addresses the research questions outlined in Section 3, provides a summary of the research and identifies the limitations of this research.

7.1 Summary of Key Contributions

RQ1 Can we implement Tara's design on Flink? Yes. Tara's nodes correspond closely to Flink's stream operators each, and the two kinds of sending that the protocol requires, both work with Flink's keyed streams. For a particular replica, Gumti also uses a key computation mechanism to make sure that Flink's hashing assigns a key to that replica for a particular message. All it sends per replica is one such copy to reach all replicas. The only exception are the feedback loops, which are not part of Flink's dataflow, but are closed outside Flink as the sinks forward their data back to the sources via the network. The entire protocol could be transferred using these building blocks, without any change of its design.

RQ2 How replica failure handle by Gumti? Is it similar like Tara? Yes. This is done at the protocol level: the controllers detect that no execution is happening, initiate a view change, and the new leader reconstructs its state from the committers' records. These experiments prove that this is the case. The system is able to come back to full speed a few seconds after each kill and hold up under a steady 4,000 requests per second.

RQ3 What replaces a Heron capability that Flink does not offer? Gaps are filled with Two small additions. The first one is a ZooKeeper instance where each replica is registered with its network address, and the replica locate the network addresses of other replicas. For connections that are defined outside of the dataflow, nothing like this exists in Flink. The second are the feedback loops: they're just regular network connections from sinks to sources, where queues were used in the original design. Both additions exist independently from the protocol, without mixing in with it, meaning that the protocol logic did not need to be modified.

Summary: The thesis shows that Tara's central idea, expressing state-machine replication as a stream-processing application, is portable: it does not depend on Heron, and it survives the move to an engine with a rather different feature set. The port required about 2,800 lines of Java, and most of these deal with attaching the protocol to the engine

and not the actual consensus itself. Based on the evaluation it appears like a system that could order up to 5300 requests per second on a single machine, with 3 ms latency floor, it can recover up to a complete failure of the leader in seconds and it appears to act as described by the protocol. On a general purpose stream engine, the evaluation reveals that the measured performance depends as much on the general behaviour of the stream engine buffers and timers as on the protocol itself.

7.2 Limitations

Single machine: All experiments are performed on a single machine with replicas being run on different task slots, using the same hardware and communicating via a local connection. This is sufficient to analyse how the protocol acts, but the actual numbers, such as the latencies would be different in an actual network.

Only leader failures tested: We only test failure of the active proposer, since if the leader fails then the full view change machinery is used, etc. Loss of replicas of other stages, such as committers, executors etc., was not tested, but the protocol is tolerant to the loss of one replica for each stage.

References

[1] Tyler Akidau, Robert Bradshaw, Craig Chambers, Slava Chernyak, Rafael J Fernández-Moctezuma, Reuven Lax, Sam McVeety, Daniel Mills, Frances Perry, Eric Schmidt, et al. The dataflow model: a practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. *Proceedings of the VLDB Endowment*, 8(12):1792–1803, 2015.

[2] Paris Carbone, Gyula Fóra, Stephan Ewen, Seif Haridi, and Kostas Tzoumas. Lightweight asynchronous snapshots for distributed dataflows. *arXiv preprint arXiv:1506.08603*, 2015.

[3] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, Kostas Tzoumas, et al. Apache flink™: Stream and batch processing in a single engine. *IEEE Data Eng. Bull.*, 38(4):28–38, 2015.

[4] Miguel Castro, Barbara Liskov, et al. Practical byzantine fault tolerance. In *OsDI*, volume 99, pages 173–186, 1999.

[5] Michael Isard, Mihai Budiu, Yuan Yu, Andrew Birrell, and Dennis Fetterly. Dryad: distributed data-parallel programs from sequential building blocks. In *Proceedings of the 2nd ACM SIGOPS/EuroSys European conference on computer systems 2007*, pages 59–72, 2007.

[6] Sanjeev Kulkarni, Nikunj Bhagat, Maosong Fu, Vikas Kedigehalli, Christopher Kellogg, Sailesh Mittal, Jignesh M Patel, Karthik Ramasamy, and Siddarth Taneja. Twitter heron: Stream processing at scale. In *Proceedings of the 2015 ACM SIGMOD international conference on Management of data*, pages 239–250, 2015.

[7] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. In *Concurrency: the Works of Leslie Lamport*, pages 179–196. 2019.

[8] Leslie Lamport et al. Paxos made simple. *ACM Sigact News*, 32(4):18–25, 2001.

[9] Laura Lawniczak and Tobias Distler. Stream-based state-machine replication. In *2021 17th European Dependable Computing Conference (EDCC)*, pages 119–126. IEEE, 2021.

[10] Leonardo Neumeyer, Bruce Robbins, Anish Nair, and Anand Kesari. S4: Distributed stream computing platform. In *2010 IEEE International Conference on Data Mining Workshops*, pages 170–177. IEEE, 2010.

[11] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *2014 USENIX annual technical conference (USENIX ATC 14)*, pages 305–319, 2014.

[12] Fred B Schneider. Implementing fault-tolerant services using the state machine approach: A tutorial. *Acm Computing Surveys (CSUR)*, 22(4):299–319, 1990.

[13] The Apache Software Foundation. Powered by Flink. `https://flink.apache.org/what-is-flink/powered-by/`, 2026. Accessed: 2026-06-28.

[14] Ankit Toshniwal, Siddarth Taneja, Amit Shukla, Karthik Ramasamy, Jignesh M Patel, Sanjeev Kulkarni, Jason Jackson, Krishna Gade, Maosong Fu, Jake Donham, et al. Storm@ twitter. In *Proceedings of the 2014 ACM SIGMOD international conference on Management of data*, pages 147–156, 2014.

[15] Matei Zaharia, Tathagata Das, Haoyuan Li, Timothy Hunter, Scott Shenker, and Ion Stoica. Discretized streams: Fault-tolerant streaming computation at scale. In *Proceedings of the twenty-fourth ACM symposium on operating systems principles*, pages 423–438, 2013.

In accordance with § 9 Para. 12 APO, I hereby declare that I wrote the preceding master's thesis independently and did not use any sources or aids other than those indicated.
I also certify that I have utilized the following AI tools: ChatGPT, Claude, and Gemini during the thesis. While I was preparing, I only used these tools responsibly and in a limited and supportive way. I also used these tools to check grammar and to plan notes as I was creating paper summaries. I am confirming that no AI-generated text has been directly copied or included in this thesis. All ideas, interpretations and written material are my own.
Furthermore, I declare that the digital version corresponds without exception in content and wording to the printed copy of the master's thesis and that I am aware that this digital version may be subjected to a software-aided, anonymised plagiarism inspection.

Bamberg, 03.08.2026 Md Moinuddin